2026-08-18-auto-committer-BOB-068-investigation

Incident 2026-08-18 — BOB-068 “auto-committer sweep pattern” Phase-1 investigation

Revision: 1 **Last modified:** 2026-08-18T21:32:00Z **Reporter:** conductor (SDD task-41 dispatch, 2026-08-18) **Session-under-investigation:** 4db6eadb-03e7-466b-9cb4-34b2b2bd30f3 (boba SDD orchestration) **Investigator:** Phase-1 systematic-debugging subagent (sonnet), task-41 **Class:** parallel-subagent shared-checkout race (§11.4.84 working-tree quiescence / §9.2 data-safety hazard class) **Terminal state:** ROOT CAUSE CONFIRMED — no daemon exists; race condition on shared checkout + shared SSoT file, verdict (b) per task brief **Iron Law compliance:** Phase 1 (root cause investigation) ONLY. No fix proposed or written in this document — see §7 for the disposition of the already-drafted remedy.

1. What was investigated

Multiple parallel-subagent reports this session (task #55, task #66, task #68, and the §11.4.179 proposal doc, `docs/proposals/subagent-worktree-isolation.md`) describe a recurring symptom: **a subagent’s scoped, intentional commit lands folded together with — or entirely superseded by — a concurrent sibling subagent’s commit**, because both subagents were writing into the **same shared git checkout** and the **same shared `docs/workable_items.db` SSoT file** at the same time. The task brief for this investigation named it “the BOB-068 auto-committer sweep pattern” and asked two concrete questions: (1) does an actual auto-committer daemon exist anywhere on this host, and (2) if not, is this a race condition on `git add -A` (or equivalent) in a non-isolated shared checkout.

Terminology note (found during investigation, recorded honestly per §11.4.6): the label “BOB-068” is used for **two distinct things** in this session’s own documents. See §3 below — this is itself a finding, not an assumption this report makes.

2. Evidence gathered — is there an auto-committer daemon?

All checks below were run directly against the live host and the live `boba` git checkout during this investigation. Full commands and raw output are in the session transcript; the results are summarized here.

2.1 systemd timers (user scope; `--sudo` access is hard-blocked on this host per

Hard Stop §6.U — see 2.6)

```
$ systemctl --user list-timers --all
```

9 timers listed. 8 are transient `podman healthcheck run <container-id>` units (this project’s OWN qbittorrent/jackett/boba-jackett container healthchecks, per `docker-compose.yml`) plus the standard `systemd-tmpfiles-clean.timer`. **None invoke git, none reference a repository path, none contain “commit”/“push”/“sync” in their `ExecStart`.** Inspected one representative unit directly:

```
$ systemctl --user cat a91ba43aff...-42f265332bed8851.service
ExecStart="/usr/bin/podman" "healthcheck" "run"
"a91ba43aff157e66fd31294357b81823a2858a72a690fb25191fa37b4547f8c8"
```

Confirmed: container healthcheck, unrelated to git.

2.2 cron

```
$ crontab -l
no crontab for milosvasic
```

`/etc/crontab` and `/etc/cron.d/` are root-owned and unreadable without `sudo` (permission denied) — but per §2.6 below, `sudo`/`su` in any tool call is **mechanically blocked** on this host by the project’s own `guard-forbidden-commands.sh` PreToolUse hook, which is itself evidence the host enforces no-root-side-channels for agent-driven investigation. No user-level cron job exists; a root-level `/etc/cron.d` entry auto-committing to a project the interactive user owns would be an extreme outlier inconsistent with every other finding in this investigation (see §2.3–§2.5), and no other evidence anywhere in this investigation is consistent with one.

2.3 git hooks

```
$ ls -la .git/hooks/
total 52
(only *.sample files — no non-sample hook present)

$ git config --get core.hooksPath
(empty — unset, default .git/hooks/ in effect)
```

No installed git hook of any kind (pre-commit, post-commit, pre-push, etc.) exists in this checkout. `core.hooksPath` is unset, so there is no redirected/external hook directory either.

2.4 Editor / IDE auto-commit settings

```
$ grep -iE "git\.(autofetch|postCommit|autoStash|autoPush)|
autoSave|commit" ~/.config/Code/User/settings.json
(no output — no matches)
```

No project-root `.vscode/settings.json` exists (only `frontend/.vscode/`, an Angular subproject IDE folder, out of scope for repo-root git automation and not consulted by git itself). No VSCode git-automation setting is configured globally either.

2.5 Running processes

```
$ pgrep -af "commit|watch|sync|inotify"
```

Matches, all confirmed unrelated to git-commit automation on manual inspection:

- `kworker/R-sync_wq` — a Linux **kernel** worker thread for the `sync_wq` workqueue (page-cache/filesystem sync, not "git sync").
- `watchdogd` — the systemd hardware watchdog daemon (host liveness, unrelated).
- A CodeGraph indexer watchdog subprocess (`@colbymchenry/codegraph-linux-x64`) — its own inline source, printed verbatim in the process list, shows it exists solely to `SIGKILL` a wedged CodeGraph indexing process after a silence timeout; it never touches git.
- A `chrome-devtools-mcp` telemetry watchdog — unrelated to git or file commits.
- The two `pgrep`-invoking shell processes from this very investigation command.

No process anywhere on the host matches a git-commit-automation signature (no `gitwatch`, no `entr`-driven commit loop, no cron-spawned script, no daemon holding a `.git` lock open).

2.6 Historical “Auto-commit”-named commits — checked and ruled out as daemon evidence

Two bare-message `Auto-commit` commits exist in this repo’s history (`54e313f`, 2026-08-08; `c10ad07`, 2026-06-10 — plus the 18 more the `docs/QA_DISCOVERY_LEDGER.md` BOB-068/RD2-00 entry already enumerates: `9c8f684`, `743097a`, `de9270b`, `1c36777`, `41179c2`, `7c529ca`, etc.). These are **not** evidence of a daemon on **this** investigation’s scope:

- Both inspected commits (`54e313f`, `c10ad07`) carry the **identical author/committer identity** (`Милош Васић <i@mvasic.ru>` / `Milos Vasic <i@mvasic.ru>`) as every other commit in this repository — no bot account, no service identity, no distinguishing committer.
- The pre-existing `docs/QA_DISCOVERY_LEDGER.md` RD2-00 entry (filed 2026-08-08, this project’s own prior root-cause pass — not authored by this investigation) already traced the **newer** wave of these commits via `git reflog` to an ordinary **`git pull --ff` fast-forward from a second live session/host** with push access to the same remotes — evidenced by a commit timezone (`+0500`) that does not match the investigating host’s own (`+0300` at the time). That is a **second human/agent session operating a different clone**, not an automated daemon on this host.
- The `guard-forbidden-commands.sh` PreToolUse hook (§2.6 above, and confirmed live during this very investigation when it correctly BLOCKED a `sudo`-containing command) demonstrates this host actively enforces mechanical guardrails against exactly the class of unattended privileged automation a “daemon” would require — consistent with none existing.

Conclusion of §2: no auto-committer daemon exists on this host, in this checkout, or in any configuration file, hook, timer, or process inspectable from this session. Verdict (a) from the task brief (“daemon exists at path X”) is **refuted** by direct evidence.

3. Two different things both called “BOB-068” this session (finding, not an

assumption)

The tracked workable item **`BOB-068`** in `docs/workable_items.db` / `docs/Issues.md` (line 44) is titled *“RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main”* and its full description is specifically about the **historical, cross-host bare-`Auto-commit`-message** phenomenon investigated in §2.6 above — root-caused (in a prior, 2026-08-08 session, not this one) to a second

live session/host fast-forward-pulling in commits, **not** a daemon, and **still tracked as `Queued`** because the mechanical guard for it (filed as followup `BOB-106`) has not yet been authored.

This session's own newly-authored documents — `.superpowers/sdd/task-66-brief.md` ("BOB-068 pattern discovered 5x this session: commit-push-all.sh unconditionally does `git add -A` which sweeps in-flight parallel-subagent work into unrelated commits"), the commit message of `0972cbc` ("`commit-push-all.sh --scope` flag ends BOB-068 sweep pattern"), and `docs/proposals/subagent-worktree-isolation.md`'s own §1 Problem Statement ("BOB-068 identified a real defect class: parallel SDD subagents dispatched against this project's single shared checkout… can commit… directly onto `main` with no structural isolation") — all **reuse the `BOB-068` label** to refer to a **different, newly-observed defect class from THIS session**: parallel SDD subagents racing on a shared checkout, not the historical cross-host pull phenomenon.

This is a real labeling inconsistency in this session's own tracker/doc trail (the tracked `BOB-068` item's description does not match what three of this session's own documents call "the BOB-068 pattern"), noted here for the record. **This investigation proceeds on the defect class the task brief actually asked about** — the shared- checkout sweep pattern — since that is what "5+ occurrences this session" and the cited Task #55/#66/#68 evidence are all describing. Reconciling the DB item's title/description with this session's actual usage (or filing the sweep pattern under its own correctly-titled item) is an open item, tracked in §8, not fixed here (Phase 1 scope).

4. Evidence gathered — is this a race condition on a shared, non-isolated checkout?

4.1 No git-level isolation exists between subagents dispatched this session

```
$ ls -la .git/worktrees/
ls: cannot access '.git/worktrees/': No such file or directory

$ git worktree list
/run/media/milosvasic/DATA4TB/Projects/boba  98412bf [main]
```

Exactly **one** working tree exists for the entire session: the single checkout at `/run/media/milosvasic/DATA4TB/Projects/boba`. No `git worktree add`, no `git clone --local`, and no container-per-subagent mechanism was used for any subagent dispatched this session.

Every subagent operated directly inside this one shared `.git`, one shared index, one shared working tree — confirmed by direct inspection, not inferred.

4.2 `scripts/commit-push-all.sh`'s own committed comments confirm the mechanism

```
$ grep -n "flock\|git add\|lock" scripts/commit-push-all.sh
...
30: # Without --scope, stage 5 runs an unconditional `git add -A`,
which
...
39: # a stray `git add -A` from an earlier/concurrent run left
residue),
...
109:LOCK="$(git rev-parse --git-dir)/.commit_push_all.lock"
111:if command -v flock >/dev/null 2>&1; then
112:    flock -n 9 || {
...
222:    git add -- "${SCOPES[@]}"
...
257:    git add -A
```

The script has **always** (both before and after this session's Task #66 interim fix) had an `flock`-protected critical section keyed on `$(git rev-parse --git-dir)/.commit_push_all.lock`. This **serializes invocations of the script itself** — two `commit-push-all.sh` processes cannot literally execute their commit logic concurrently. It does **not**, and structurally cannot, serialize or isolate the **file-level edits** subagents make to the shared working tree *between* script invocations. Whichever invocation acquires the lock first stages whatever the shared working tree looks like **at that instant** — before Task #66's fix, unconditionally (`git add -A`, line 257); the flag added by Task #66 makes an explicit `--scope` list (line 222) the alternative, but the **unscoped `git add -A` remains the default** for backward compatibility (confirmed in the script's own committed comment, line 30, and in `commit-push-all.sh`'s CLI: callers that do not pass `--scope` still get `git add -A`).

4.3 Direct, first-person evidence of the sweep from this session's own subagent

`reports`

Task #55's report (`.superpowers/sdd/task-55-report.md`, "Concerns" §1, written by the affected subagent itself, not this investigation):

> A concurrent sibling subagent (working on task #68 / `BOB-108`…) was independently running `close`/`export` against the exact same shared `docs/workable_items.db` in this same non-isolated checkout at the same time. My scoped commit landed with "nothing to commit" on its second invocation because the sibling's own `commit-push-all.sh` run had already committed and pushed the working-tree state (which by then included my `BOB-104` update) as part of commit `3520621`.

Task #68's report (`.superpowers/sdd/task-68-report.md`, "Concerns"):

> `docs/workable_items.db`'s tracked bytes grew by a few KB purely from SQLite's own read-path/WAL housekeeping across the several `export`/`validate`/`close` invocations this session… Issues.md's regeneration also materialised BOB-109 through BOB-114 (already resident in the DB, filed by other work this session, not yet synced into the file before this fix ran). This is the correct, intended §11.4.93 behaviour of `export`, not scope creep.

Commit `3520621` (BOB-108 closure) itself confirms this in its own message: "a few bytes of SQLite housekeeping (WAL/page-cache churn from opening the file across export+validate+close invocations this session; meta table content is unchanged)."

4.4 Timing evidence — commits landing seconds apart, consistent with lock-

```
serialized queuing of near-simultaneous subagent completions

91b52db 2026-08-18T21:02:50+02:00  fix(boba-jackett,BOB-112): TTL
cache /healthz ...
0972cbc 2026-08-18T21:02:51+02:00  feat(scripts,#66): commit-push-
all.sh --scope flag ...
c7dfdde 2026-08-18T21:02:53+02:00  feat(dev-tools,BOB-113):
install-dev-tools.sh ...
eb22c50 2026-08-18T21:02:54+02:00  docs(qa,evidence):
session-2026-08-16 test suite ...
```

Four commits from (per their subject lines) four **different** work items land within a 4-second window, each with author date == committer date (no rebase/backdating). This is the observable signature of the `flock` doing exactly what §4.2 describes: multiple subagents' `commit-push-all.sh` invocations queued on the same lock and fired in rapid succession as each acquired and released it — each one's `git add`/`git add -A` capturing a snapshot of the shared tree at that moment, which is why a file touched by more than one concurrently-running subagent (most concretely, `docs/`

`workable_items.db` and its derived exports) shows up bundled into whichever commit happens to run first, as documented directly by the affected subagents in §4.3.

4.5 The `--scope` interim fix (Task #66) narrows one vector but does not — and `structurally cannot — close the shared-SSoT-file vector`

Task #66's own report and brief are explicit that `--scope` is an **opt-in, interim** mitigation for the `git add -A` vector specifically (arbitrary unrelated dirty files being swept in). It does **not** address, and cannot address, the second vector directly demonstrated in §4.3: `docs/workable_items.db` (the §11.4.93/§11.4.95 single- source-of-truth database) and its derived exports (`docs/Issues.md`, `docs/Fixed.md`, etc.) are **one physical file each**, mutated in place by every subagent's `workable-items` CLI invocations in the **same shared checkout**. Task #68's own commit used **precise, non-`-A` staging** ("used precise `git add <files>`, never `git add -A`" — task-68-report.md, "Scope discipline") and its commit **still** legitimately and correctly captured BOB-109 through BOB-114 — entries filed by *other, concurrently-running subagents* — because the `export` tool, by design, regenerates the whole document from the DB's **current, shared** state at the moment it runs. There is no git-staging-layer fix for this: the sweep, in this specific sub-case, happens at the **data layer** (one shared DB file with no per-subagent isolation), not at the git-staging layer `--scope` operates on.

5. Verdict

(b) — No daemon. Confirmed race condition, with two distinct contributing mechanisms, both rooted in the SAME underlying architectural gap: subagents dispatched this session share ONE git checkout and ONE `docs/workable_items.db` SSoT file, with no filesystem- or git-level isolation between them.

1. **Git-staging-layer vector:** `commit-push-all.sh`'s default (still-unscoped) `git add -A` sweeps any file dirty in the shared tree at the moment a subagent's invocation acquires the serializing `flock`, regardless of which subagent produced the dirty state. (Mitigated, not eliminated, by Task #66's opt-in `--scope` flag — the unscoped default remains for backward compatibility.)
2. **Shared-data-layer vector:** even precise, scoped `git add` of exactly the intended files still captures **whatever the shared `docs/workable_items.db` (and its derived exports) currently contain**, because that file is the SAME physical file every concurrently-dispatched subagent's `workable-items` CLI writes to. No git-staging discipline can separate "my mutation" from

> “a sibling’s concurrent mutation” when both land in one shared file before either subagent commits.

Both vectors trace to the identical root: **no per-subagent isolation (no git worktree, no clone, no container) exists for this session’s SDD subagent dispatch pattern** — every subagent runs directly against the conductor’s own single checkout.

6. Cross-check against the already-drafted §11.4.179 proposal

`docs/proposals/subagent-worktree-isolation.md` (authored earlier this session, Task #67, independently of this investigation) reaches the **same root-cause conclusion** in its own §1 Problem Statement, arrived at via a separate investigative pass: “parallel SDD subagents dispatched against this project’s single shared checkout… can commit… directly onto `main` with no structural isolation between concurrent subagents’ in-flight work.” That proposal additionally measured (§3–§5 of that document, real commands run on this host) that `git worktree add` does **not** satisfy §11.4.179’s corruption-isolation requirement (shared `.git` object store / shared lock namespace), while `git clone --local` (hardlinked objects, same filesystem) does, at near-identical cost (0.19s vs. 0.22s measured).

This investigation’s independently-gathered evidence (§2–§4 above) **confirms** that proposal’s problem statement is accurate and that its proposed remedy — genuine per-subagent `.git` isolation via `git clone --local`, with reconciliation via a real `fetch → merge → resolve → push` at “reap” time (§11.4.113) rather than a shared, last-writer-wins working tree — is the correct architectural fix for **both** vectors identified in §5: per-subagent clones would give each subagent its own physical copy of `docs/workable_items.db`, so a genuine conflicting mutation between two subagents would surface as a **real, reviewable merge conflict at reap time** (per §11.4.211, Fable-`xhigh` conflict resolution) instead of a silent blend attributed to whichever subagent’s commit-script invocation happened to win the `flock` race.

7. Disposition (Phase 1 boundary — no fix written here)

Per the Iron Law and this task’s explicit scope, **no fix is proposed or written in this document.** The correct remedy has already been independently designed and measured (§6, `docs/proposals/subagent-worktree-isolation.md`, §11.4.179) and is **confirmed correct by this investigation’s evidence**. That proposal’s own

§9 (“Explicit non-implementation notice”) and §10 (“Open items”) already state, honestly, that nothing has been implemented yet — this investigation does not change that; it adds independent confirmation that the proposal targets the right root cause.

8. Open items (tracked, not silently dropped — §11.4.197)

- Implement `multitrack_subagent_clone.sh` (spawn/reap/list) per `docs/proposals/subagent-worktree-isolation.md` §6 — the architectural fix for both vectors identified in §5. Not attempted here (out of this task’s Phase-1-only scope).
- Reconcile the `BOB-068` tracked-item title/description (historical cross-host `Auto-commit` phenomenon, §3) against this session’s informal reuse of the same label for the shared-checkout sweep pattern — either retitle/re-scope `BOB-068` to match its actual DB description and file the sweep pattern under its own correctly named item, or formally fold the sweep pattern into `BOB-068`’s tracked scope. Left unresolved by this investigation (documentation-consistency item, not a Phase-1 root-cause question).
- `BOB-106` (mechanical guard for the *historical* cross-host unattributed-commit phenomenon, §2.6/§3) remains separately open and unaffected by this investigation.
- The pre-existing `BOB-010` evidence-path violation surfaced incidentally by `workable-items validate` during this session (noted in both task-55 and task-68 reports) is confirmed pre-existing and out of scope for this investigation.

9. Evidence index

- `.superpowers/sdd/task-55-report.md` — first-person account of the sweep from the affected subagent.
- `.superpowers/sdd/task-66-brief.md` / commit `0972cbc` — the interim `--scope` remedy and its own before/after characterization.
- `.superpowers/sdd/task-68-report.md` — first-person account showing precise (non-`-A`) staging still captures concurrent siblings’ data via the shared SSoT file.
- `docs/proposals/subagent-worktree-isolation.md` — the independently-drafted architectural remedy, cross-checked and confirmed by this investigation.
- `docs/QA_DISCOVERY_LEDGER.md` (RD2-00/BOB-068 entry) — the historical, distinct cross-host phenomenon (§3).
- `scripts/commit-push-all.sh` (read-only inspection in this investigation; not modified) — the `flock` serialization + `git add -A` default confirmed at lines 109–257.

- Direct host commands run in this session (systemd timers, crontab, git hooks, git config, process list, git worktree list) — full output in the session transcript backing §2 and §4.1.